02 / LEARNING GUIDE

Python Reference Guide

EX28 · NumPy: 2D arrays, transpose and matrix products

> **Reading order:** 01 Engineering & Math Theory → 02 Python Reference → 03 Exercise → 04 Pseudocode → programming. This guide belongs to EX28, class 18.

1. Importing NumPy and creating arrays

We import `numpy` as `np`. A 2D array contains a list of rows; each row is another list. In this exercise, all rows have the same length.

```
import numpy as np

A = np.array([[1, 2],
              [3, 4]], dtype=float)
x = np.array([2, -1], dtype=float)
```

The `dtype=float` argument selects floating-point values. It is optional here but is common in scientific computing.

2. Array dimensions

Expression	Returned value	Example
A.shape	Tuple (rows, columns)	(2, 2)
A.ndim	Number of axes	2
A.size	Number of entries	4
x.shape	Shape of 1D array	(2,)

These are array **attributes**, not function calls: write `A.shape`, not A.shape(). An array of shape (2,) is not identical to a 2 × 1 matrix.

3. Entry-wise operations

```
B = np.array([[2, 0],
              [-1, 3]], dtype=float)

S = A + B        # Addition: equal shapes
D = A - B        # Subtraction
K = 2.0 * A      # Scale all entries
H = A * B        # Element-wise multiplication
```

Inputs: equally shaped arrays for these examples. Output: a new array with the same shape. NumPy also supports broadcasting; this is outside the scope of EX28.

4. Transpose

```
AT = A.T

print(A.shape)   # (2, 2)
print(AT.shape)  # (2, 2)
```

`A.T` provides a transposed view for an ordinary NumPy array. `np.transpose(A)` is an alternative. A 2 × 3 array becomes 3 × 2.

5. Matrix multiplication with @

```
y = A @ x        # (2,2) @ (2,) -> (2,)
C = A @ B        # (2,2) @ (2,2) -> (2,2)
```

The @ operator implements matrix multiplication and corresponds to `np.matmul` for these cases. Shapes of the inputs determine whether the product is valid.

$$A\boldsymbol{x} = \begin{pmatrix} 0 \\ 2 \end{pmatrix}$$

The crucial difference: * versus @

```
A * B  # Element-by-element multiplication
A @ B  # Row-by-column matrix product
```

These expressions are **not interchangeable**. `np.dot` also resembles @ for simple 1D and 2D inputs, but higher-dimensional behavior can differ. Here we prefer @ because it explicitly identifies matrix multiplication.

6. Column arrays versus 1D arrays

```
x = np.array([2, -1])
x_col = np.array([[2], [-1]])

print(x.shape)      # (2,)
print(x_col.shape)  # (2, 1)
```

They contain the same scalar components but **have different shapes**. We will use 1D vectors in this class to keep the code simple.

7. Clear output formatting

```
print(f"Matrix shape: {A.shape}")
print(f"Transposed matrix:\n{A.T}")
print(f"Result vector: {A @ x}")
```

The `\n` sequence starts a new line. Include engineering units in physical results, not in purely algebraic outputs.

8. Common issues

Situation	What to check
Unexpected A.shape	Count rows and columns
A @ x gives ValueError	Inner dimensions do not match
Unexpected result from A * B	Use @ for matrix multiplication
A.T looks the same as A	Is A symmetric or a 1D array?

> **EX28 scope:** NumPy, one Python file, manual checks. No JSON, pytest, custom package structure, or inverse linear-system solving.